

Recursion and Stack - Simple Hinglish Summary

Source: <https://javascript.info/recursion>

Main baat

Ab functions ko aur gehrai se dekhte hain. Pehla topic: **recursion**.

Recursion ek programming pattern hai jo tab kaam aata hai jab:

- Kaam **usi type ke kai chhote kaamo** me baant sakta ho, ya
- Kaam **ek aasan action + usi kaam ka ek aasan version** me simplify ho sakta ho, ya
- Kuch data structures ke saath kaam karna ho.

Jab function kisi kaam ko solve karta hai to wo kai dusre functions call kar sakta hai. Iska ek special case: **function khud ko call kare**. Ise **recursion** kehte hain.

1. Sochne ke do tarike

Ek function `pow(x, n)` likhte hain jo `x` ko natural power `n` par le jaye (yaani `x` ko khud se `n` baar multiply kare):

javascript

```
pow(2, 2) = 4
pow(2, 3) = 8
pow(2, 4) = 16
```

Tarika 1: Iterative (loop se)

javascript

```
function pow(x, n) {
  let result = 1;

  // result ko x se n baar multiply karo
  for (let i = 0; i < n; i++) {
    result *= x;
  }

  return result;
}

alert( pow(2, 3) ); // 8
```

Tarika 2: Recursive (kaam simplify karo aur khud ko call karo)

javascript

```
function pow(x, n) {  
  if (n == 1) {  
    return x;  
  } else {  
    return x * pow(x, n - 1);  
  }  
}  
  
alert( pow(2, 3) ); // 8
```

Recursive variant bunyadi taur par alag hai. $\text{pow}(x, n)$ call hone par execution do branches me batata hai:

$$\text{pow}(x, n) = \begin{cases} \text{if } n==1 & = x \\ / \\ \backslash \\ \text{else} & = x * \text{pow}(x, n - 1) \end{cases}$$

1. $n == 1$ **ho to** sab trivial hai. Ise recursion ka **base** kehte hain, kyunki ye turant obvious result deta hai: $\text{pow}(x, 1) = x$.
2. **Warna** $\text{pow}(x, n)$ ko $x * \text{pow}(x, n - 1)$ likh sakte hain. Ise **recursive step** kehte hain: kaam ko ek aasan action (x se multiply) aur usi kaam ke ek aasan call (n kam karke) me badla. Agle steps ise aur simplify karte hain jab tak n 1 na ho jaye.

$\text{pow}(2, 4)$ ke steps:

1. $\text{pow}(2, 4) = 2 * \text{pow}(2, 3)$
2. $\text{pow}(2, 3) = 2 * \text{pow}(2, 2)$
3. $\text{pow}(2, 2) = 2 * \text{pow}(2, 1)$
4. $\text{pow}(2, 1) = 2$

Yaani recursion function call ko ek aasan call me, phir aur aasan me badalti hai, jab tak result obvious na ho jaye.

Recursion aksar chhoti hoti hai. Conditional operator $?$ se aur terse:

javascript

```
function pow(x, n) {  
  return (n == 1) ? x : (x * pow(x, n - 1));  
}
```

Recursion depth: nested calls ki maximum ginti (pehla call samet). Yahan ye exactly n hai.

Maximum recursion depth JS engine tay karta hai. Hum lagbhag **10000** par bharosa kar sakte hain. Kuch engines zyada dete hain, lekin **100000** zyadatar ke liye limit se bahar hai. Kuch automatic optimizations ("tail call optimization") madad karte hain, lekin wo har jagah supported nahi aur sirf simple cases me kaam karte hain.

Isse recursion ka use limited hota hai, lekin phir bhi bahut kaam ka hai. Kai tasks me recursive soch se simple, maintain karne me aasan code milta hai.

2. Execution context aur stack

Ab dekhte hain recursive calls kaise kaam karte hain. Iske liye functions ke andar jhankte hain.

Chalte hue function ki execution ki jaankari uske **execution context** me store hoti hai.

Execution context ek internal data structure hai jisme function ki execution ki details hoti hain: control flow abhi kahan hai, current variables, `this` ki value, aur kuch internal details.

Ek function call ka ek hi execution context hota hai.

Function jab nested call karta hai to:

- Current function **pause** hota hai.
- Uska execution context ek special data structure me **yaad rakha jata hai**, jise **execution context stack** kehte hain.
- Nested call chalta hai.
- Wo khatam hone ke baad purana execution context stack se wapas liya jata hai, aur bahar wala function wahin se **resume** hota hai jahan ruka tha.

`pow(2, 3)`

Call shuru hone par execution context me variables: `x = 2, n = 3`, execution flow function ki line `1` par.

Context: { x: 2, n: 3, at line 1 } pow(2, 3)

`n == 1` falsy hai, isliye flow `if` ki doosri branch me jata hai:

javascript

```
function pow(x, n) {
  if (n == 1) {
    return x;
  } else {
    return x * pow(x, n - 1);
  }
}
```

Variables wahi, line badli:

Context: { x: 2, n: 3, at line 5 } pow(2, 3)

`x * pow(x, n - 1)` calculate karne ke liye naye arguments ke saath subcall `pow(2, 2)` chahiye.

`pow(2, 2)`

Nested call ke liye JS current execution context ko **execution context stack** me yaad rakhta hai.

Yahan wahi function `pow` call ho raha hai, lekin isse fark nahi padta. Process har function ke liye same hai:

1. Current context stack ke **upar yaad** rakha jata hai.
2. Subcall ke liye **naya context** banta hai.
3. Subcall khatam hone par pichhla context stack se **nikala** jata hai aur uski execution aage badhti hai.

`pow(2, 2)` me jaane par stack:

Context: { x: 2, n: 2, at line 1 } pow(2, 2)	<- ab chal raha hai (upar)
Context: { x: 2, n: 3, at line 5 } pow(2, 3)	<- yaad rakha hua

Subcall khatam hone par pichhla context wapas lena aasan hai, kyunki usme variables aur code ki wo exact jagah dono hain jahan wo ruka tha.

(Ek line me kai subcalls bhi ho sakte hain, jaise `pow(...) + pow(...) + somethingElse(...)`). Isliye zyada sahi ye kehna hai ki execution "**subcall ke turant baad**" se resume hoti hai.)

`pow(2, 1)`

Process dohrata hai: line `(5)` par naya subcall, ab `(x=2)`, `(n=1)` ke saath. Naya context banta hai, pichhla stack ke upar push hota hai:

```
Context: { x: 2, n: 1, at line 1 } pow(2, 1)
Context: { x: 2, n: 2, at line 5 } pow(2, 2)
Context: { x: 2, n: 3, at line 5 } pow(2, 3)
```

Ab 2 purane contexts hain aur 1 abhi `pow(2, 1)` ke liye chal raha hai.

Exit (bahar aana)

`pow(2, 1)` me is baar `n == 1` truthy hai, isliye `if` ki pehli branch chalti hai. Koi nested call nahi, isliye function khatam hota hai aur `2` return karta hai.

Function khatam hone par uska context memory se **hata diya** jata hai. Pichhla stack ke upar se wapas aata hai:

```
Context: { x: 2, n: 2, at line 5 } pow(2, 2)
Context: { x: 2, n: 3, at line 5 } pow(2, 3)
```

`pow(2, 2)` resume hota hai. Uske paas subcall `pow(2, 1)` ka result hai, isliye wo `x *`
`pow(x, n - 1)` poora karke `4` return karta hai.

Phir pichhla context wapas aata hai:

```
Context: { x: 2, n: 3, at line 5 } pow(2, 3)
```

Ye khatam hone par `pow(2, 3) = 8`.

Is case me recursion depth **3** thi. Recursion depth stack me maximum contexts ki ginti ke barabar hoti hai.

Memory ka dhyan: contexts memory lete hain. Yahan power `n` par le jane ke liye `n` contexts ki memory chahiye (saari chhoti values ke liye).

Loop-based algorithm zyada memory-saving hai: iterative `pow` ek hi context use karta hai jisme sirf `i` aur `result` badalte hain. Iski memory chhoti, fixed aur `n` par depend nahi karti.

Koi bhi recursion loop me badli ja sakti hai. Loop variant aksar zyada effective ban sakta hai.

...Lekin kabhi rewrite mushkil hota hai, khaaskar jab function conditions ke hisaab se alag recursive subcalls use kare aur unke results merge kare, ya branching complex ho. Aur optimization ki zarurat na ho to koshish worth nahi. **Recursion chhota, samajhne me aasan aur support karne me aasan code de sakti hai.** Har jagah optimization zaruri nahi, hume achha code chahiye, isliye recursion use hoti hai.

3. Recursive traversals (recursively chalna)

Recursion ka ek aur zabardast use: **recursive traversal**.

Maan lo ek company hai. Staff structure object se dikha sakte hain:

```
javascript

let company = {
  sales: [{
    name: 'John',
    salary: 1000
  }, {
    name: 'Alice',
    salary: 1600
  }],

  development: {
    sites: [{
      name: 'Peter',
      salary: 2000
    }, {
      name: 'Alex',
      salary: 1800
    }],

    internals: [{
      name: 'Jack',
      salary: 1300
    }]
  }
};
```

Yaani company ke **departments** hain.

- Ek department me **staff ka array** ho sakta hai. Jaise `sales` me 2 employees.
- Ya department **subdepartments** me baant sakta hai. Jaise `development` ke do branches: `sites` aur `internals`, jinke apne staff hain.
- Subdepartment badhne par wo **subsubdepartments (teams)** me bhi baant sakta hai. Jaise `sites` aage `siteA` aur `siteB` me. Aur wo aur bhi.

Ab hume saari salaries ka sum nikalne wala function chahiye. Kaise?

Iterative tarika aasan nahi, kyunki structure simple nahi. Pehla idea: `company` par `for` loop aur pehle level ke departments par nested subloop. Phir `sites` jaise 2nd level

departments ke staff ke liye aur subloops... phir 3rd level ke liye aur. Ek object traverse karne me 3-4 nested subloops code ko **bhaddha** bana dete hain.

Recursion try karte hain. Function ko department milne par do cases hote hain:

1. Ya to **"simple" department** jisme logo ka **array** hai. Tab simple loop se salaries jod sakte hain.
2. Ya wo **(N) subdepartments wala object** hai. Tab har subdepartment ke liye recursive call karke sum lo aur results jodo.

1st case recursion ka base hai (trivial case, jab array mile). **2nd case recursive step hai** (object mile to kaam chhote departments ke subtasks me baant do). Wo aage aur baant sakte hain, lekin der-saber (1) par khatam honge.

javascript

```
let company = { // wahi object, chhota karke
  sales: [{name: 'John', salary: 1000}, {name: 'Alice', salary: 1600 }],
  development: {
    sites: [{name: 'Peter', salary: 2000}, {name: 'Alex', salary: 1800 }],
    internals: [{name: 'Jack', salary: 1300}]
  }
};

// kaam karne wala function
function sumSalaries(department) {
  if (Array.isArray(department)) { // case (1)
    return department.reduce((prev, current) => prev + current.salary, 0); // a
  } else { // case (2)
    let sum = 0;
    for (let subdep of Object.values(department)) {
      sum += sumSalaries(subdep); // subdepartments ke liye recursively call, r
    }
    return sum;
  }
}

alert(sumSalaries(company)); // 7700
```

Code chhota aur samajhne me aasan hai. Ye recursion ki power hai. Ye **subdepartments ki kisi bhi nesting** ke liye chalta hai.

Principle: **(...)** object ke liye subcalls hote hain, jabki arrays **[...]** recursion tree ke **"patte" (leaves)** hain, jo turant result dete hain.

Code me pehle seekhi cheezein use hui: `arr.reduce` (array ka sum), aur `for(val of Object.values(obj))` (object ki values par iterate).

4. Recursive structures

Recursive (recursively-defined) data structure wo structure hai jo khud ko apne hisson me dohrata hai.

Ye humne upar company structure me dekha: Company ka **department** ya to:

- **Logo ka array**, ya
- **Departments wala object**.

Web developers ke liye behtar jaane-pehchane examples: **HTML aur XML documents**. HTML document me ek *HTML-tag* me ye ho sakte hain:

- Text ke tukde
- HTML-comments
- Dusre *HTML-tags* (jinme phir text/comments/tags ho sakte hain)

Ye bhi recursive definition hai.

Behtar samajh ke liye ek aur recursive structure: "**Linked list**", jo kuch cases me arrays ka behtar alternative ho sakta hai.

Linked list

Maan lo objects ki **ordered list** store karni hai. Natural choice **array** hai:

```
javascript
let arr = [obj1, obj2, obj3];
```

Lekin arrays me ek problem: "**delete element**" aur "**insert element**" mehngi operations hain. Jaise `arr.unshift(obj)` ko naye `obj` ke liye jagah banane ke liye **saare elements renumber** karne padte hain. Array bada ho to time lagta hai. `arr.shift()` ke saath bhi yahi.

Bina mass-renumbering wale structural modifications sirf **array ke end** par hote hain: `arr.push/pop`. Isliye bade queues me jab shuru par kaam karna ho to array dheera ho sakta hai.

Alternative: agar tez insertion/deletion chahiye to dusra data structure **linked list**.

Linked list element recursively define hota hai ek aise object ki tarah jisme:

- `value`
- `next` property, jo agle linked list element ko refer karti hai, ya `null` agar end ho.

```
javascript
```

```

let list = {
  value: 1,
  next: {
    value: 2,
    next: {
      value: 3,
      next: {
        value: 4,
        next: null
      }
    }
  }
};

```

Banane ka alternative code:

javascript

```

let list = { value: 1 };
list.next = { value: 2 };
list.next.next = { value: 3 };
list.next.next.next = { value: 4 };
list.next.next.next.next = null;

```

Yahan aur saaf dikhta hai ki **kai objects** hain, har ek me `value` aur padosi ki taraf `next`. `list` variable chain ka **pehla object** hai, isliye `next` pointers follow karke kisi bhi element tak pahunch sakte hain.

List ko aasani se kai hisson me tod aur wapas jod sakte hain:

javascript

```

let secondList = list.next.next;
list.next.next = null;

```

Jodne ke liye:

javascript

```

list.next.next = secondList;

```

Aur kisi bhi jagah insert ya remove kar sakte hain.

Shuru me naya value jodna: list ka head update karo:

javascript

```
let list = { value: 1 };  
list.next = { value: 2 };  
list.next.next = { value: 3 };  
list.next.next.next = { value: 4 };  
  
// naya value list ke shuru me jodo  
list = { value: "new item", next: list };
```

Beech se value hatana: pichhle wale ka `next` badlo:

javascript

```
list.next = list.next.next;
```

`list.next` ne ① ko **kood kar (jump)** ② par pahunch gaya. ① ab chain se bahar hai. Agar wo kahin aur store nahi hai to memory se apne aap hat jayega.

Arrays ke ulta, **mass-renumbering nahi**, elements ko aasani se rearrange kar sakte hain.

Lists hamesha arrays se behtar nahi hote. Nahi to sab sirf lists hi use karte. **Main nuksan:** number se element aasani se nahi mil sakta. Array me `arr[n]` seedha reference hai. List me pehle item se shuru karke `next` ko `N` baar follow karna padta hai.

...Lekin aise operations hamesha nahi chahiye. Jaise queue ya **deque** (ordered structure jisme dono ends se bahut tez add/remove chahiye, beech tak pahunch ki zarurat nahi).

Lists ko enhance kar sakte hain:

- `next` ke saath `prev` property jodo, taaki peeche jana aasan ho.
- Aakhri element ko refer karne wala `tail` variable jodo (end se add/remove karte waqt update karo).
- ...Data structure zarurat ke hisaab se badal sakta hai.

Summary

Terms:

- **Recursion:** function ko khud se call karna. Recursive functions se tasks elegant tarike se solve ho sakte hain.
 - Function khud ko call kare to use **recursion step** kehte hain.
 - **Recursion ka basis:** function ke aise arguments jinse kaam itna simple ho jaye ki function aage calls na kare.

- **Recursively-defined data structure:** aisa data structure jo khud se define ho sake. Jaise linked list: ek object jo ek list (ya `null`) ko refer karta hai.

```
javascript
```

```
list = { value, next -> list }
```

- **Trees** (HTML elements ka tree, ya department tree) natural taur par recursive hain: unki branches hain aur har branch me aur branches ho sakti hain. Unhe chalne ke liye recursive functions use kar sakte hain (`sumSalaries` example).

Koi bhi recursive function iterative me badla ja sakta hai. Aur kabhi optimization ke liye ye zaruri bhi hota hai. Lekin kai tasks ke liye recursive solution kaafi tez hota hai aur likhna aur support karna aasan.

Practice Tasks (Answers)

1. Diye gaye number tak saare numbers ka sum (`sumTo`): `sumTo(4) = 4 + 3 + 2 + 1 = 10`.

Teen variants:

Loop se:

```
javascript
```

```
function sumTo(n) {
  let sum = 0;
  for (let i = 1; i <= n; i++) {
    sum += i;
  }
  return sum;
}
```

Recursion se (`sumTo(n) = n + sumTo(n-1)`):

```
javascript
```

```
function sumTo(n) {
  if (n == 1) return 1;
  return n + sumTo(n - 1);
}
```

Formula se (arithmetic progression: `n*(n+1)/2`):

```
javascript
```

```
function sumTo(n) {
  return n * (n + 1) / 2;
}
```

Kaun sa sabse tez? Formula sabse tez hai (kisi bhi n ke liye sirf 3 operations). **Loop** doosre number par. **Recursion** sabse dheeri (nested calls aur stack management me resources lagte hain).

Kya `sumTo(100000)` recursion se ho sakta hai? Aam taur par **nahi**. Zyadatar engines tail call optimization support nahi karte, isliye "**maximum stack size exceeded**" error aata hai.

2. Factorial (`factorial(n)`): $n! = n * (n-1) * (n-2) * \dots * 1$. Yaani $n! = n * (n-1)!$

javascript

```
function factorial(n) {
  return (n !== 1) ? n * factorial(n - 1) : 1;
}

alert( factorial(5) ); // 120
```

Base 1 hai. 0 ko bhi base bana sakte hain:

javascript

```
function factorial(n) {
  return n ? n * factorial(n - 1) : 1;
}
```

3. Fibonacci numbers (`fib(n)`): $F_n = F_{n-1} + F_{n-2}$. Sequence: `1, 1, 2, 3, 5, 8, 13, 21...`

Recursive (par dheera):

javascript

```
function fib(n) {
  return n <= 1 ? n : fib(n - 1) + fib(n - 2);
}

alert( fib(3) ); // 2
alert( fib(7) ); // 13
// fib(77); // bahut dheera hoga!
```

Kyu dheera? Function bahut zyada subcalls karta hai. Wahi values baar baar evaluate hoti hain. Jaise `fib(3)` `fib(5)` aur `fib(4)` dono ke liye chahiye, isliye do baar alag alag calculate hota hai. Calculations `n` se bahut tezi se badhte hain.

Behtar: loop se (dynamic programming bottom-up):

```
javascript

function fib(n) {
  let a = 1;
  let b = 1;
  for (let i = 3; i <= n; i++) {
    let c = a + b;
    a = b;
    b = c;
  }
  return b;
}

alert( fib(3) ); // 2
alert( fib(7) ); // 13
alert( fib(77) ); // 5527939700884757
```

Loop `i=3` se shuru hota hai kyunki pehle do values `a=1`, `b=1` hard-coded hain. Har step me sirf pichhli do values yaad rakhni padti hain.

(Doosra tarika: pehle se calculate ki hui values **yaad rakhna (memoization)**.)

4. Single-linked list output karo (`printList`):

Loop se:

```
javascript

function printList(list) {
  let tmp = list;

  while (tmp) {
    alert(tmp.value);
    tmp = tmp.next;
  }
}
```

`list` parameter ko sidha badalne ki jagah temporary variable `tmp` use karna behtar hai, taaki future me `list` par kuch aur karna ho to wo na khoye.

Recursion se:

javascript

```
function printList(list) {  
  
    alert(list.value); // current item output  
  
    if (list.next) {  
        printList(list.next); // baaki list ke liye wahi karo  
    }  
  
}
```

Kaunsa behtar? Technically **loop zyada effective** hai (nested calls ke resources nahi lagte). Recursive variant **chhota** hai aur kabhi samajhne me aasan.

5. List ko ulte order me output karo (printReverseList):

Recursion se (pehle baaki list output karo, **phir** current):

javascript

```
function printReverseList(list) {  
  
    if (list.next) {  
        printReverseList(list.next);  
    }  
  
    alert(list.value);  
}
```

Loop se: list me aakhri value seedha nahi mil sakti aur "peeche" nahi ja sakte. Isliye pehle seedhe order me items array me yaad rakho, phir ulte order me output karo:

javascript

```
function printReverseList(list) {
  let arr = [];
  let tmp = list;

  while (tmp) {
    arr.push(tmp.value);
    tmp = tmp.next;
  }

  for (let i = arr.length - 1; i >= 0; i--) {
    alert( arr[i] );
  }
}
```

Recursive solution asal me yahi karta hai: list ko follow karta hai, items ko nested calls ki chain (execution context stack) me yaad rakhta hai, aur phir output karta hai.

Quick Summary

Baat	Yaad rakho
Recursion	Function ka khud ko call karna
Base case	Wo case jahan function aage call nahi karta (turant result)
Recursive step	Kaam ko ek aasan action + chhote call me badalna
Recursion depth	Nested calls ki maximum ginti (aam taur par ~10000 tak)
Execution context	Ek function call ki jaankari (variables, line, <code>this</code>)
Execution context stack	Nested calls me pichhle contexts yahan yaad rakhe jate hain
Memory	Recursion (n) contexts leti hai, loop sirf ek
Recursive structure	Khud ko dohrane wala (linked list, HTML tree, departments)
Tree traversal	Recursion sabse aasan (array = leaf, object = recursive step)
Koi bhi recursion	Loop me badli ja sakti hai

Array vs Linked list:

	Array	Linked list
Shuru me insert/remove	Dheera (renumbering)	Tez
Number se element (<code>arr[n]</code>)	Tez	Dheera (<code>next</code>) <code>N</code> baar)

Yaad rakho:

- Recursion me **base case zaruri** hai, warna function kabhi rukega nahi (stack overflow).
- Recursion **chhoti aur saaf** hoti hai, lekin loop **zyada memory-efficient**.
- `fib(n)` jaise cases me recursion ek hi cheez baar baar calculate karti hai, loop ya memoization behtar hai.